Great Learning

6-7 minutes

As LLM-based applications evolve from simple prompt–response systems to autonomous, long-running assistants powered by AI Agents, the way we orchestrate reasoning, memory, and decision flow becomes a core architectural concern. A poor orchestration choice can lead to brittle systems that are hard to debug, impossible to scale, or unsafe to deploy in production.

While there are multiple tools available for designing, building, and orchestrating AI Agents, two of the most common, widely-used Python libraries are LangChain and LangGraph.

This page will provide an introduction to both LangChain and LangGraph, and discuss the practical differences and use case suitability for both. In the hands-on videos and case studies of this and upcoming weeks, we'll be using these libraries to design, build, and orchestrate AI Agents to solve a variety of business problems.

Introduction

LangChain: Sequential and Pipeline-Oriented Design

LangChain is built around the concept of chains, where each step in a workflow executes in a predefined order. The output of one component becomes the input to the next, forming a linear or directed acyclic pipeline. This model aligns well with traditional software pipelines and deterministic execution flows.

The framework abstracts common tasks such as prompt management, memory handling, tool invocation, and agent execution. However, context propagation is largely transient and scoped to a single execution flow. Once a chain completes, there is no inherent mechanism to preserve or evolve state across multiple runs.

From a theoretical standpoint, LangChain treats LLM-powered applications as stateless or short-lived workflows, making it suitable for predictable, one-shot or limited multi-step interactions.

LangGraph: Graph-Based and State-Centric Design

LangGraph adopts a fundamentally different theoretical model. Instead of linear chains, it represents workflows as graphs composed of nodes and edges. Each node represents a computational unit (such as an LLM call, tool execution, or decision logic), while edges